tcb/box tcb/boxSect
tcb/box/begin0 tcb/box/begindefault
tag=tcb/box tcb/box/begindefault
tcb/box/end0 tcb/box/enddefault
tcb/box/enddefault
tcb/box/title tcb/box/titleCaption
tcb/box/title0 tcb/box/titledefault para/semantictcb/box/title tcb/box/titledefault
tcb/box/draw2 tcb/box/drawdefault 2
tcb/box/drawdefault
tcb/box/init0 tcb/box/initdefault minipage/beforenoopminipage/afternoop tcb/box/initdefault
tcb/box/upper0 tcb/box/upperdefault minipage/beforetag/dfltminipage/aftertag/dflt tcb/box/upperdefault
tcb/box/lower0 tcb/box/lowerdefault minipage/beforetag/dfltminipage/aftertag/dflt tcb/box/lowerdefault
tcb/drawing/init0tcb/drawing/initdefaulttcb/drawing/initdefault

Post-Quantum Blockchains with Agility in Mind

Abstract. The abstract should briefly summarize the contents of the paper in 150–250 words.

Keywords: First keyword · Second keyword · Another keyword.

MANEL Manuel's comments
DANNO Danno's comments
RON Ron's comments
MIKE Mike's comments
FIXME **Please fix this**
TODO There is a todo here

1 Introduction

FIXME **Add references.**

Blockchains provide decentralized trust through cryptographic verification rather than reliance on trusted third parties. Unlike centralized systems where data can be modified or revoked, blockchains are designed to be persistent and immutable, providing long-term integrity guarantees. Their security relies on multiple building blocks, including the consensus layer, execution layer, transaction mechanisms, and peer-to-peer communication protocols. These components are built on cryptographic primitives such as hash functions (e.g., SHA-256, Keccak-256) and digital signature schemes. The security of these primitives is therefore foundational to blockchain integrity [**?**].

Historical precedent shows that cryptographic primitives inevitably become vulnerable due to algorithmic advances (e.g., MD5, SHA-1), implementation flaws, or paradigm shifts such as quantum computation. Given the long-term lifespan of blockchains, systems must be able to transition cryptographic primitives without disrupting operation. This ability, referred to in NIST CSWP 39 as cryptographic agility [**?**], is essential for sustaining security over time. However, the properties that make blockchains valuable also create challenges when cryptographic primitives must be updated, replaced, or deprecated, making cryptographic agility a fundamental design feature.

The most immediate threat to blockchain cryptography comes from quantum computers. Current systems rely heavily on elliptic curve signature schemes such as ECDSA and EdDSA, which are vulnerable to cryptographically relevant quantum computers via Shor's algorithm. Despite this risk, migration to post-quantum cryptography remains slow. A survey of over 14,000 blockchain users found that although quantum threats are widely acknowledged, 58.8% stated

that post-quantum security is "not important" when choosing chains or applications [?]. This perception gap, combined with coordination challenges, deployment processes that risk network forks, and long-term backward compatibility requirements for dormant accounts, significantly delays transition efforts.

These challenges help explain why major blockchain systems such as Bitcoin and Ethereum currently lack cryptographic agility, and why platforms with partial agility support, including Hedera and Cosmos [?], still struggle with comprehensive post-quantum transitions. Existing analysis, such as NIST WP 39 [?], also lacks an overview of current approaches and best practices to achieve crypto agility in open participation decentralized systems. This paper addresses these gaps by designing a blockchain system with cryptographic agility as a foundational principle from genesis, enabling secure cryptographic transitions as threats evolve while preserving the persistence and immutability that define blockchains.

1.1 Our Contribution

We design a post-quantum secure blockchain with cryptographic agility to allow future migrations.
RON testss

1.2 Related Work

FIXME **Add references**

The blockchain industry has recognized the quantum threat for over a decade, prompting various high-profile projects to adopt distinct approaches to post-quantum security. We examine existing efforts, categorizing them into chains requiring migration and post-quantum-native solutions.

Existing Chains Requiring Migration. Solana introduced the Winternitz Vault in January 2025 [?], a sophisticated construction based on the Winternitz One-Time Signature (WOTS) [?]. The vault mechanism employs programmatic address derivation to compress signatures, achieving post-quantum security while mitigating blockspace overhead. However, this opt-in, application-layer solution creates a fragmented security landscape where only proactive users gain protection while the majority remain vulnerable. Moreover, vaults must be instantiated through standard transactions that themselves rely on quantum-vulnerable EC-based signatures, leaving the bootstrapping process exposed.

Algorand pioneered post-quantum implementations among major blockchains, introducing State Proofs in 2022—compact, post-quantum secure certificates signed by validator supermajorities using FALCON signatures [?,?,?]. While State Proofs represent a significant achievement, other aspects of Algorand continue to rely on quantum-vulnerable cryptography: transaction signatures use EC cryptography, and the consensus mechanism depends on a pre-quantum verifiable random function (VRF) [?]. Since Algorand launched without post-quantum security from genesis, its transition process faces the coordination and migration difficulties inherent to protocol-wide cryptographic upgrades.

Bitcoin's migration discourse centers on several community proposals under active discussion. The prominent draft by Lopp [?] outlines a phased approach: Phase A would prohibit transactions to legacy quantum-vulnerable address types (P2PK, P2PKH) three years after BIP-360 implementation, forcing new UTXOs into quantum-resistant formats (P2QRH) [?]. Phase B would invalidate transactions from legacy addresses after a fixed period, freezing unmigrated funds. However, these approaches have met with skepticism within the community. The viability of such migrations depends heavily on the BIP-32 wallet standard, creating problems for non-conforming wallets such as paper wallets.

Ethereum has adopted a comprehensive research approach through multiple initiatives [?,?]. For user-level transaction signing, post-quantum migration remains under heavy research without concrete implementation plans. While Account Abstraction (EIP-4337) [?] has been proposed as a potential enabling technology [?], theoretical proposals involving STARK proofs face significant practical challenges including substantial computational overhead and unclear migration paths. At the consensus layer, Ethereum researchers investigate quantum-resistant alternatives for BLS signatures in Proof-of-Stake [?]. Despite this theoretical sophistication, Ethereum's approach faces enormous coordination challenges affecting every layer of the protocol stack, requiring consensus among globally distributed stakeholders.

BCR25 [?] presents a migration strategy for EdDSA-based chains with backward compatibility. However, this approach lacks the cryptographic agility framework necessary for future transitions beyond the initial post-quantum migration.

Existing Post-Quantum Chains and Solutions. Several academic works propose blockchain transition schemes. ICCCN:SM17 [?] proposes a scheme inspired by long-term signature schemes, but assumes availability of a secure timestamping service and a trusted public key infrastructure (PKI), requiring decentralization of the PKI system with critical practical consequences and added governance issues. ESORICS:MenChe22 [?] further develops and formalizes the transition scheme, but does not take into account API and data format details in blockchain implementations, presenting transitions only at a conceptual level addressing hash and signature schemes rather than specific blockchain subsystems. EPRINT:FM25 [?] highlights migration issues but only mentions generic migration paths.

Among existing chains, QRL (Quantum Resistant Ledger) uses XMSS (eXtended Merkle Signature Scheme) for post-quantum security, implementing a cap on the number of signatures generated and an address format that allows for different signature schemes. However, QRL is not an EVM-based chain, limiting its compatibility with the broader Ethereum ecosystem. QANplatform incorporates Dilithium signatures for cross-chain signing, but this addresses only a narrow aspect of post-quantum security rather than the comprehensive stack-wide problems. The platform lacks cryptographic agility in its design, focusing on a single post-quantum primitive without migration mechanisms. The Quantus design addresses post-quantum security in both P2P node communication and transaction layers. However, Quantus follows a proof-of-work consensus mechanism, whereas

our approach uses proof-of-stake, creating fundamental architectural differences in how consensus and security are achieved. Neptune Cash implements post-quantum cryptography but lacks modularity and does not incorporate cryptographic agility or migration strategies in its design, making future transitions difficult.

2 Preliminaries

Preliminaries: with subsections regarding the tools we are using. In this case we can introduce whatever cryptographic material we rely on, whatever consensus mechanism we use, the cryptographic agility premises

2.1 Post-Quantum Cryptography

KEMs.

Signatures.

2.2 Blockchain Building Blocks

Transactions RLP format

Discv4 The packet encoding follows this structure:

$$\begin{aligned}
\texttt{payload} &= \texttt{packet_type} \parallel \mathsf{RLP}(\texttt{message_data}) \\
\texttt{message_hash} &= \mathsf{keccak256}(\texttt{payload}) \\
(\texttt{signature}, \texttt{recovery_id}) &\leftarrow \mathsf{ECDSA_sign_recoverable}(\texttt{message_hash}, sk) \\
\texttt{sig_bytes} &= \texttt{signature} \parallel \texttt{recovery_id} \parallel \texttt{payload} \\
\texttt{hash} &= \mathsf{keccak256}(\texttt{sig_bytes}) \\
\texttt{Packet} &= \texttt{hash} \parallel \texttt{sig_bytes}
\end{aligned} \tag{1}$$

where `signature` is 64 bytes (the (r, s) pair), `recovery_id` is 1 byte, and `hash` is 32 bytes. Upon receiving a packet, a node verifies the signature by recovering the public key from `message_hash`, `signature`, and `recovery_id`, then derives the `node_id` from the recovered public key. The packet structure is:

$$\texttt{Packet} = \underbrace{\texttt{hash}[32]}_{\text{header}} \parallel \underbrace{\texttt{signature}[64] \parallel \texttt{recovery_id}[1] \parallel \texttt{packet_type}[1] \parallel \mathsf{RLP}(\texttt{message_data})}_{\text{payload}} \tag{2}$$

3 Cryptographic Flexibility in Blockchains

Cryptographic agility is a design principle that enables systems to adapt their cryptographic algorithms over time without disrupting their live functioning or requiring fundamental architectural changes [?]. In this section, we introduce the principles established by NIST Cybersecurity White Paper (CSWP) 39 [?] to enable effective cryptographic agility and highlight the unique challenges blockchains face in adopting them. We then present a flexibility framework that characterizes how algorithm choice can be distributed across blockchain participants, providing a lens through which to analyze the design space for cryptographic transitions.

3.1 Challenges

The NIST Cybersecurity White Paper (CSWP) 39 [?] establishes a set of principles that enable effective cryptographic agility. Among others, it stresses the need for algorithm identification through ciphersuite negotiation and naming conventions; modularity that separates cryptographic algorithms from application logic, enabling algorithms to be treated as interchangeable components; policy and mechanism separation that allows cryptographic policies to be stored in configuration files rather than hard-wired in source code; and mechanisms for preserving interoperability during transitions. Additionally, systems must handle the deployment of algorithm transitions, which can take years when new infrastructure is required, and manage cryptographic keys associated with deprecated algorithms through proper key expiration and revocation procedures.

While the NIST principles provide a foundation for cryptographic agility in traditional systems, each concept faces unique challenges when applied to blockchain environments. More specifically, ciphersuite negotiation, which works effectively in point-to-point protocols like TLS [?], becomes impractical among an unbounded number of distributed nodes. Modularity is limited when protocols depend on algorithm-specific properties rather than using cryptography as a black box. For instance, KZG [?] commitments are tied to specific data types, and BLS [?] signatures are chosen specifically for their aggregation properties, making drop-in replacements non-trivial. Moreover, preserving interoperability presents asymmetric challenges between blockchain entities, as different participants face different coordination constraints (discussed in Section ??). Key lifecycle management is particularly problematic in blockchains, where addresses serve as long-lived identities tied to deprecated algorithms, making key expiration and revocation difficult to implement without disrupting user accounts. Finally, deploying algorithm transitions in consensus scenarios is more complex than in centralized systems. Indeed, not all nodes upgrade simultaneously, making it difficult to determine which nodes are honest, potentially leading to forks and increased vulnerability to 51% attacks during transition periods.

3.2 Flexibility Spectrum

The challenges outlined in Section **??** demonstrate that blockchain systems cannot adopt traditional cryptographic agility principles wholesale. Instead, cryptographic agility in blockchains is best understood as a design space where different components require different degrees of flexibility based on their coordination constraints.

Coordination Constraints. Different blockchain participants face different pressures when cryptographic algorithms change. Node operators can coordinate quickly: when consensus-critical vulnerabilities emerge, the economic cost of non-compliance (missed rewards, slashing, chain splits) creates strong incentives for rapid, synchronized upgrades. The patching of CVE-2018-17144 within days [**?**] exemplifies this dynamic. For these stakeholders, hard transitions at defined upgrade points are operationally feasible. However, even for node operators, hardware adoption can present significant barriers. Many hardware security modules (HSMs) lack support for certain post-quantum algorithms (e.g., Falcon), and operational constraints such as long HSM purchasing and refresh cycles can slow down cryptographic migration. This makes it necessary to support multiple signature schemes to accommodate different business and hardware timelines, even among participants who are otherwise willing to coordinate.

End users, by contrast, interact with the chain sporadically and cannot be expected to upgrade in lockstep. A user who returns after two years should not discover that their funds are inaccessible because they failed to migrate during a transition window. Legacy wallets may remain incompatible with new cryptographic schemes indefinitely. For user-facing components, the system must therefore support multiple algorithms simultaneously, allowing gradual adoption without forcing coordinated action.

These coordination asymmetries, between those who operate the network and those who use it, determine which flexibility strategies are feasible for each blockchain layer.

Flexibility Models. We characterize cryptographic agility along a *flexibility spectrum* that captures how algorithm choice is distributed across participants. This spectrum ranges from rigid enforcement of a single algorithm to full user-side choice, with hybrid and negotiated models in between.

- *Singular.* Exactly one algorithm is permitted system-wide at any point in time. All participants must use the same algorithm, which simplifies verification logic but creates a single point of cryptographic failure: if that algorithm is broken, the entire system is compromised. This model suits components where all parties can coordinate upgrades synchronously.
- *Layered.* Multiple algorithms are composed in series, providing defense-in-depth during periods of cryptographic uncertainty. Security holds unless *all* layers are simultaneously broken. This model is appropriate when confidence

in any single algorithm is insufficient, particularly during transitions to post-quantum cryptography.

- *Negotiated.* Both communicating parties select an algorithm from the intersection of their individually supported sets. This approach is standard in point-to-point protocols but difficult to generalize to decentralized systems where transactions must be validated by an unbounded set of nodes without prior negotiation.
- *Selected.* One party unilaterally chooses an algorithm from a protocol-defined allowed set. This shifts flexibility to the edge where users can adopt new algorithms at their own pace. The tradeoff is that verifiers must maintain implementations of all supported algorithms. *Example:* CATX transactions (Section **??**), where signers specify their signature scheme and verifiers accept any scheme from the allowed set.

Mapping Blockchain Components. Based on the flexibility spectrum and the coordination constraints of each layer, we propose the following positioning for our design.

Consensus Layer: Singular. Consensus requires all validators to agree on block validity using identical verification logic. Supporting multiple signature algorithms would complicate fork-choice rules and increase the attack surface during transitions. We therefore adopt a singular model where all validators use the same algorithm (Falcon-512 in our implementation). Upgrades occur via hard forks at predefined block heights, leveraging validators' strong coordination incentives. Section **??** details the migration mechanism for transitioning between singular algorithms.

P2P Communication: Layered. The peer-to-peer layer establishes secure channels between nodes. Because P2P participants are node operators who can coordinate upgrades, a hard transition to new algorithms is feasible. However, given current uncertainty about post-quantum algorithm maturity, we adopt a layered (hybrid) approach using X-Wing for key encapsulation. This provides immediate post-quantum protection while retaining classical security as a fallback. Section **??** presents the KEM-based protocols adopted.

Transaction Layer: Selected. Transactions originate from users who cannot be forced to upgrade synchronously. Legacy wallets must remain functional, and users should be able to adopt post-quantum signatures at their own pace. We therefore adopt a selected model: senders choose their signature algorithm from an allowed set, and all validators must support verification of every algorithm in that set. The CATX transaction format (Section **??**) enables this by encoding algorithm identifiers directly in the transaction structure, decoupling signature schemes from transaction semantics.

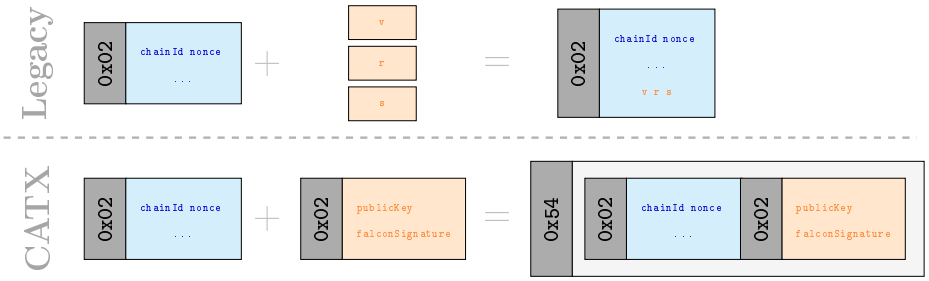

Fig. 1. Legacy vs. CATX transaction structure. Legacy transactions (top) embed ECDSA signature fields directly into the transaction body, whereas CATX (bottom) separates the transaction payload from the signature container within a Type `0x54` wrapper.

Execution Layer (Precompiles): Selected. On-chain cryptographic operations exposed via precompiled contracts follow the same selected model as transactions. Smart contracts invoke precompiles with algorithm-specific inputs, and the execution layer must support all algorithms in the allowed set. This enables contracts to adopt new cryptographic primitives without requiring protocol upgrades for each addition.

4 CATX

How does it support future migrations? Ref Figure TODO

Traditional Ethereum transaction formats have a fundamental barrier to cryptographic agility through their deep integration of ECDSA signatures into the transaction structure. The signature components, specifically the recovery parameter `v` and the signature values `r` and `s`, are directly embedded within the RLP-encoded transaction payload along with its body data (e.g., `chainId`, `nonce`) as exemplified below with EIP-1559 (Type `0x02`):

$$\texttt{Tx} = \texttt{0x02} \parallel \mathsf{RLP}([\texttt{chainId}, \texttt{nonce}, \ldots, \texttt{v}, \texttt{r}, \texttt{s}]) \tag{3}$$

This design creates several limitations that prevent cryptographic agility. First, the transaction format assumes a public key recovery mechanism. This represents a concrete case of the modularity challenge identified in Section **??**: the transaction protocol is not designed to use cryptographic primitives in a black box manner, but rather depends on specific properties of ECDSA (namely, public key recovery) that are baked into the transaction structure itself.

Second, the signature structure is hardcoded into the transaction serialization format, violating the modularity principle discussed in Section **??**. Unlike point-to-point protocols such as TLS, where cryptographic algorithms can be negotiated at runtime through ciphersuite selection, blockchain transactions must be validated by an unbounded set of distributed nodes without prior negotiation.

This distributed validation requirement, as discussed in Section **??**, makes runtime algorithm negotiation impractical and necessitates that the cryptographic algorithm be encoded directly in the transaction format.

CATX Design: Separating Body from Signature To address the above issues, we present Cryptographically Agile Transactions (CATX), which introduces transaction type `0x54` that achieves cryptographic modularity. CATX design maintains the same core transaction items and base signing object as traditional Ethereum transactions, but package it into a self-contained blob. The core crypto agile innovation lies in separating the transaction body from its signature, enabling independent evolution of transaction semantics and cryptographic primitives.

CATX structure can be described as follows:

$$\texttt{Tx} = \texttt{0x54} \parallel \texttt{bodyType} \parallel \mathsf{RLP}(\texttt{bodyData}) \parallel \texttt{sigType} \parallel \mathsf{RLP}(\texttt{sigData}). \tag{4}$$

This design choice separates concerns into four independent components: body type (`bodyType`), body data (`bodyData`), signature type (`sigType`) and signature data (`sigData`).

The body type identifies the transaction body format, allowing for multiple transaction body formats and each identified by a unique body type value (e.g., `0x02` for fee market EIP-1559 transactions). The body data contains the transaction semantics (e.g., chain ID, nonce, gas parameters). This body type registry allows new transaction formats to be added without affecting signature handling. The signature type identifies the cryptographic signature algorithm, enabling pluggable cryptographic schemes. This follows the crypto agility norm of algorithm identification [**?**], where an identifier is used for an implementation to tell the verification peer which algorithm is being used. For our experiments, we set the following algorithm identities: `0x01` for ECDSA-secp256k1, `0x02` for FALCON-512 and `0x03` for ML-DSA-44. Finally, the signature data contains the algorithm-specific signature material. For ECDSA, the signature data contains the standard three parameters (`v`, `r`, `s`). Post-quantum signatures might have support for extractable public key from a signature. Depending on the signature type supported, specified by `sigType`, the verification precompile input pattern accepts two variants: (`pubKey` ‖ `signature` ‖ `digest`) or (`extractableSignature` ‖ `digest`).

Message digest. Post-quantum signature algorithms typically support signing messages of arbitrary length [**?**]. At first glance, it may therefore seem natural to sign the transaction data directly. However, for consistency with the ECDSA-based design, we instead adopt a message-digest approach, since ECDSA only accepts 32-byte inputs. Concretely, under CATX the signed message is

$$\texttt{digest} = \mathsf{Keccak256}\left(\texttt{bodyType} \parallel \mathsf{RLP}(\texttt{bodyData})\right). \tag{5}$$

This design choice offers two main advantages. First, it preserves compatibility with the existing ECDSA workflow, improving interoperability with current

tooling and enabling signatures to be treated as a black box, independent of algorithm-specific details. Second, in the presence of large transactions, signing the full message can significantly impact certain signing environments (e.g., KMS), whereas a fixed-size digest avoids these limitations.

Address derivation. For externally owned accounts (EOAs) that do not rely on ECDSA and instead use post-quantum signature schemes, a mechanism is required to map a public key to an address. To support multiple signature schemes while preserving cryptographic agility and enabling smooth migration, address derivation follows a structure similar to that used for ECDSA-based accounts. However, to reduce the risk of hash collisions and to clearly separate concerns across different signature types, a signature identifier (`sigType`) is appended to the public key prior to hashing. Concretely, addresses are derived as

$$\texttt{address} = \mathsf{Keccak256}\,(\texttt{sigType} \parallel \texttt{pubKey})\,. \tag{6}$$

4.1 Precompiles in Execution Layer

Beyond externally owned accounts (EOAs), Ethereum supports a growing number of smart contract wallets that rely on EVM execution logic for authentication. Account abstraction proposals such as ERC-4337 [?] enable smart contracts to define custom signature verification logic, opening a potential path for post-quantum cryptography: contracts could implement PQC verification directly in EVM bytecode.

However, the EVM is highly inefficient for the low-level, repetitive arithmetic required by post-quantum signature schemes. Benchmarks [?] show that pure-EVM verification of Falcon-512 signatures consumes approximately 6.6 million gas, while ML-DSA-44 verification requires approximately 13.5 million gas [?].

To address this limitation, we expose post-quantum signature verification as precompiled contracts, reducing gas costs by orders of magnitude through native execution. Backup with experimetnal data The precompile interface follows the pattern established by existing Ethereum cryptographic precompiles (e.g., `ecrecover` at address `0x01`), accepting the public key, signature, and message as input: TODO

$$\texttt{MLDSA44VERIFY}(\texttt{pubKey}, \texttt{signature}, \texttt{message}). \tag{7}$$

The argument ordering differs from ECDSA's `ecrecover` for two reasons. First, post-quantum schemes do not necessarily support public key recovery from signatures, making the public key a required input. Second, NIST post-quantum algorithms support signing arbitrary-length messages directly [?], so the message appears last in the argument list to allow variable-length inputs. For consistency with ECDSA workflows, many implementations still sign over a hash digest, but the precompile interface accommodates both patterns.

To conform with the algorithm identification principle for CA, the precompile addresses align with the CATX signature type identifiers: `0x5400xx`, where

`xx` corresponds to the signature type (e.g., `0x540001` for ECDSA, `0x540002` for Falcon-512, `0x540003` for ML-DSA-44). A new ECDSA precompile is also provided at `0x540001` with the unified argument layout, enabling smart contracts to handle all signature types through a consistent interface. This design supports the *selected* flexibility model: smart contract wallets can verify any supported signature algorithm, allowing users to choose their preferred PQC scheme without requiring contract upgrades.

5 Cryptographic Agile Consensus

Consensus layers usually enforce a fixed cryptographic signature scheme set at genesis, with all nodes validating on startup to ensure configuration consistency [**?**]. This approach mirrors the singular flexibility model discussed in Section **??**: all validators must use the same algorithm at any given time, simplifying verification logic. However, embedding the scheme in genesis configuration means that changing the consensus signature algorithm requires a hard fork, creating a significant barrier to cryptographic migration.

In this section, we present an extension to tendermint-based consensus [**?**] that enables scheme changes via on-chain governance while maintaining consensus safety. The key insight is that transitions between singular schemes can be coordinated through consensus-layer transactions, allowing the network to migrate to new algorithms (e.g., Falcon-512, ML-DSA) without requiring a hard fork.

5.1 Architecture

The core contribution of this design is the introduction of *key registration transactions*—a new consensus-layer transaction type that enables validators to register new cryptographic keys for upcoming scheme upgrades. Unlike execution-layer (EVM) transactions, key registrations are processed entirely within the consensus layer. The proposer includes valid key registration transactions in the block's consensus data, and all validators process them as part of block finalization.

Figure **??** illustrates how key registrations reside in the consensus portion of the block. Validators learn all nodes' consensus keys by deterministically executing the chain state with no additional P2P gossip layer required. The supported consensus signing schemes can include different post-quantum schemes such as Falcon-512, ML-DSA-44/65/87 (FIPS 204 [**?**]), and SLH-DSA-128s (FIPS 205 [**?**]), though only one scheme is active at any given height as explained below.

5.2 Upgrade Mechanism

The upgrade process involves four phases coordinated through on-chain state. To support this process, the consensus layer maintains two new data structures that track pending upgrades and validator readiness.

Consensus Data	**Execution Data**
`height`	`ExecutionPayload`
`round`	(EVM transactions)
`proposer`	
`validatorSet`	
`keyRegistrations` ←	
`commitSignatures`	

Fig. 2. Block structure showing key registrations as consensus-layer data, separate from EVM execution.

Data structures. The consensus layer maintains a `ConsensusParams` record in chain state to track the cryptographic configuration and coordinate upgrades across all nodes. This record stores the currently active signing scheme, ensuring all validators use the same algorithm for block signatures, and a minimum upgrade delay that enforces a safety window between proposal and activation.

When governance proposes a scheme change, a `PendingSchemeUpgrade` is added to `ConsensusParams`. It stores the target activation height, the new scheme identifier, and a readiness threshold (defaulting to 6667 basis points, representing $> 2/3$ of voting power, consistent with BFT consensus thresholds [**?**]). This record allows all nodes to know that an upgrade is scheduled and track how many validators have prepared for the transition.

Validators signal their readiness by submitting a `KeyRegistrationTx`, which contains the validator's address, their new public key for the target scheme, the upgrade height, and a signature computed with their current consensus key over:

$$\mathsf{Sign}\,(\texttt{oldKey}, \texttt{"KEY_REGISTRATION"} \parallel \texttt{address} \parallel \texttt{scheme} \parallel \texttt{height} \parallel \texttt{newKey}) \tag{8}$$

Signing with the current key proves that the legitimate validator (not an impersonator) is authorizing the transition to the new key.

Upgrade flow. (1) Proposal: Governance proposes a scheme change at a future block height H, setting the `PendingSchemeUpgrade` in consensus state. External coordination is expected before this proposal, particularly for validators using hardware security modules (HSMs) that may require lead time to provision new key material. *(2) Registration:* Each validator generates a new keypair for the target scheme and submits a `KeyRegistrationTx` signed with their current key. Registration must occur at least a minimum number of blocks before activation to prevent timing attacks. *(3) Readiness check:* At height $H - 1$, the protocol verifies that sufficient voting power (exceeding the 2/3 threshold) has registered new keys. If the threshold is not met, the upgrade is cancelled. Importantly, the chain continues operating under the current scheme rather than halting. *(4) Activation:* At height H, the consensus signing scheme switches to the new algorithm. Validators who failed to register are excluded from the active validator set; they must restake and follow standard procedures to rejoin.

5.3 Security Considerations

The mechanism addresses several security threats. Timing attacks are mitigated by requiring registrations a minimum number of blocks before activation, preventing last-minute manipulation. Insufficient readiness cancels the upgrade rather than halting the chain, preserving liveness. Replay attacks are prevented by including the upgrade height in the signed registration data. Duplicate keys are rejected during validation to prevent voting power inflation.

Additionally, historical verification must be preserved after scheme changes. Old keys must be retained in chain state to verify old blocks since the deterministic replay of chain state ensures this data remains available.

6 P2P Node Communication

6.1 Execution Layer

The P2P node communication layer in Ethereum's execution layer consists of two main components: the discovery protocol (Discv4 /danno and Discv5) and the transport protocol (RLPx). The discovery phase maintains the distributed hash table (DHT) that enables nodes to find peers, while the transport protocol establishes secure communication channels between them. We explore how to improve modularity within these two systems by proposing a KEM-based P2P communication layer.

TODO Add refs

PQC Node Identity. In Ethereum, node identity `nodeId` is defined by the ECDSA signature scheme. It is defined by a hash of the corresponding public key. More specifically, it is given by $\texttt{nodeId} = \mathsf{Keccak512}(\texttt{x} \parallel \texttt{y})$, where `x` and `y` are the coordinates of the ECDSA public key. /dannoThese keys have no protocol connection to on-chain activities, This design choice ties together the P2P layer with the signature scheme being used in the consensus layer, lacking modularity and therefore increasing the number of dependencies.

In our design, we propose a KEM-based system, where `nodeId` is only dependent on the KEM keys used for P2P communication. We substitute the 64-byte public key $\texttt{x} \parallel \texttt{y}$ by a 64-byte array `peerId`, computed as

$$\mathsf{Keccak512}(\texttt{ID-HASH-DOMAIN} \parallel \mathsf{pk}_{\mathrm{KEM}}), \tag{9}$$

where `ID-HASH-DOMAIN` is a domain separator and $\mathsf{pk}_{\mathrm{KEM}}$ is the KEM public key. We denote it by `peerId` as this is the value being used by both the discovery and handshake's transfer protocol to identify nodes. We use a hash of the KEM key instead of the entire key to ensure it is smaller than a UDP discovery packet.

TODO

PQC Discovery. The Discv4 discovery protocol [?] Add ref enables nodes to discover peers through a Kademlia-based distributed hash table (DHT) maintained via UDP-based gossip messages. In the original Discv4 design, all discovery gossip messages are signed using ECDSA to provide two security features: integrity and to cryptographically bind messages to a node identity.

We propose a KEM-based Discv4 protocol without the need for signatures, while preserving the two security features required. In our approach, instead of signing the discovery gossip messages, we hash commit to the peer ID (`peerId`) and the standard message payload from the original Discv4 specification. More specifically, for all packets of type `packetType` and content `messageData` the sender computes a hash commitment scheme as follows:

$$\texttt{hash} = \mathsf{Keccak256}\big(\texttt{peerId} \parallel \texttt{packetType} \parallel \mathsf{RLP}(\texttt{messageData})\big). \tag{10}$$

Then, the sender assembles the header of the packet and its payload as follows:

$$\texttt{packet} = \underbrace{\mathsf{hash} \parallel \texttt{peerId} \parallel \texttt{packetType}}_{\text{header}} \parallel \underbrace{\mathsf{RLP}(\texttt{messageData})}_{\text{payload}}. \tag{11}$$

Upon receiving a packet, a node parses the packet according to its structure and verifies the hash commitment by locally computing it and comparing with the first parsed value.

Security. Recall from the previous section that the peer ID is derived from the hash of KEM public key. Therefore, by committing to the peer ID, the node effectively binds its identity to the message payload. Moreover, integrity is also guaranteed by the collision resistance property of the hash function, given all the message content is included as input.

Moreover, we stress that the real identity validation happens during the subsequent RLPx handshake phase, as an adversary can easily locally generat different keypairs, creating a large amount of fraudulent neighbor announcements [?]. The `peerId` received will serve as input to the handshake phase and used to check the party executing the handshake and the discovery phases is the same.

Efficiency. We observe that this approach does not have any impact on bandwidth. In fact, the final packet size is reduced by 1 byte, since the `peerId` size is the same as the signature component `r` $\parallel$ `s` and the recovery component `v` consumes one byte. Moreover, we observe that the DHT structure remains unchanged as we are using the hash of the KEM public key, making it modularity independent on the KEM type being used.

PQC RLPx Connection. The RLPx transport protocol establishes secure communication between nodes in the execution layer. In its original design, RLPx relies on ECIES (Elliptic Curve Integrated Encryption Scheme) during the handshake to derive session keys for encrypted communication.

This design exhibits limited cryptographic agility due to the tight coupling between authentication and encryption within RLPx, with no clear separation of

This protocol runs between an *initiator* and a *responder* node. The protocol establishes a secure communication channel using a KEM-based key exchange that replaces the traditional ECIES-based RLPx handshake.

- **Initiator:** Long-term KEM keypair $(\mathsf{pk}_{\text{init}}, \mathsf{sk}_{\text{init}})$. Knows the responder's peer identifier `peerID` from the discovery phase.
- **Responder:** Long-term KEM keypair $(\mathsf{pk}_{\text{resp}}, \mathsf{sk}_{\text{resp}})$.

Handshake Phase:

1. The initiator establishes a TCP connection to the responder.
2. The responder sends their public key $\mathsf{pk}_{\text{resp}}$.
3. The initiator computes $id = \mathsf{Keccak512}(\texttt{ID-HASH-DOMAIN} \parallel \mathsf{pk}_{\text{resp}})$ and checks if $id = \mathsf{peedID}$. Otherwise, abort.
4. The initiator performs KEM encapsulation, $(c_{\text{init}}, k_{\text{init}}) \leftarrow \mathsf{KEM.Enc}(\mathsf{pk}_{\text{resp}})$, generates a random nonce $n_{\text{init}} \xleftarrow{\$} \{0,1\}^{256}$ and sends the `auth` message
$$\texttt{auth} = pk_{\text{init}} \parallel c_{\text{init}} \parallel n_{\text{init}}.$$
5. The responder receives the `auth` message, performs KEM decapsulation $k_{\text{init}} \leftarrow \mathsf{KEM.Dec}(\mathsf{sk}_{\text{resp}}, c_{\text{init}})$ and performs KEM encapsulation with the initiator's public key, $(c_{\text{resp}}, k_{\text{resp}}) \leftarrow \mathsf{KEM.Enc}(\mathsf{pk}_{\text{init}})$.
6. The responder generates a random nonce $n_{\text{resp}} \xleftarrow{\$} \{0,1\}^{256}$ and sends the `ack` message $\texttt{ack} = c_{\text{resp}} \parallel n_{\text{resp}}$.
7. The initiator receives the ACK message and performs KEM decapsulation $k_{\text{resp}} \leftarrow \mathsf{KEM.Dec}(\mathsf{sk}_{\text{init}}, c_{\text{resp}})$.

Key Derivation Phase:

11. Both parties compute the key material $k = k_{\text{init}} \parallel k_{\text{resp}}$ and the nonce hash $h = \mathsf{Keccak256}(n_{\text{init}} \parallel n_{\text{resp}})$. Then, they derive the session keys as follows:
$$\begin{aligned} s_{\text{shared}} &= \mathsf{Keccak256}(k \parallel h) \\ s_{\text{aes}} &= \mathsf{Keccak256}(k \parallel s_{\text{shared}}) \\ s_{\text{mac}} &= \mathsf{Keccak256}(k \parallel s_{\text{aes}}), \end{aligned}$$
where s_{shared}, s_{aes} and s_{mac} are the shared secret, AES secret and MAC secret, respectively.

Fig. 3. RLPx handshake protocol with KEM-based key exchange. The protocol uses a bilateral KEM handshake where both parties contribute shared secrets, which are then combined to derive session keys.

concerns. Specifically, the same key pair is used both for node signatures and for asymmetric encryption during session key establishment. As a result, replacing the node signature scheme used elsewhere in the protocol stack would require a complete redesign of the RLPx handshake.

To improve modularity and cryptographic agility, we present a black-box integration of RLPx with a KEM. The resulting protocol, shown in Figure **??**,

employs a bilateral KEM handshake in which both parties contribute shared secrets that are combined to derive the session key. We instantiate the protocol with X-Wing [?], a hybrid KEM combining X25519 and ML-KEM-768.

The KEM-based RLPx handshake preserves key security properties. Shared secrets remain confidential because only the responder can decapsulate $c_{\mathbf{init}}$ and only the initiator can decapsulate $c_{\mathbf{resp}}$, ensuring session keys derived from them are protected. Also, forward secrecy is maintained by using fresh randomness in each KEM encapsulation, so compromising long-term keys does not reveal past session keys.

MIKE Do we want to talk about P2P in the consensus layer also? Its essentially identical to EL, but useful to call out. It doesn't use RLP and is used for PeerIds but serves the same purpose to authenticate validators to each other

Security. Identify sec properties and check their existence TODO

Efficiency. Add numbers on size. Point to experimental section. TODO

7 Experimental Evaluation

- **P2P Layer**
  - Number of interactions during handshake
  - Message and packet size during key exchange
  - Bandwidth overhead (classical vs. post-quantum)
  - Impact of larger keys and signatures on connection latency
- **Consensus (Malachite)**
  - EdDSA vs. Falcon signing and verification latency
  - Impact on consensus round duration
  - Signature size effects on message propagation

  DANNO For 1 would a signing only benchmark be sufficient? We will show signing is slower but verification is faster

  DANNO I can demonstrate a negative impact on blocks per second for two, although I'm not sure how to gauge message propagation. Would a grafana chart work or do we need a table?
- **Transactions (CATX)**
  - Transaction size inflation under PQ signatures
  - Block impact

  DANNO I can produce a graph/table showing number of txes witha full ECDSA sends and full ML-DSA sends.
- **Experimental Evaluation**
  - Network saturation thresholds due to signature size
  - Open-source artifacts and reproducible (?) benchmarks

  DANNO I'm not sure how I would measure number one as our metrics are not at the network layer.
- **Key Observation**
  - Post-quantum cryptography does not significantly impact computation
  - Performance degradation appears primarily when block and network size limits are reached

  /dannoI can say that the size of the signature is the dominant impact and generally speaking it's negative

8 Conclusion

References

1. Algorand: Protocol upgrade introduces state proofs for trustless cross-chain communication. Press Release (2022), https://www.prnewswire.com/news-releases/algorand-protocol-upgrade-introduces-state-proofs-for-trustless-cross-chain-communication-and-5x-html
2. Algorand: State proofs overview. Algorand Developer Portal (2024), https://developer.algorand.org/docs/get-details/stateproofs/
3. Algorand Foundation: Post-quantum technology (2024), https://algorand.co/technology/post-quantum
4. Baldimtsi, F., Chalkias, K., Roy, A.: Post-quantum readiness in EdDSA chains. Cryptology ePrint Archive, Paper 2025/1368 (2025), https://eprint.iacr.org/2025/1368
5. Barbosa, M., Connolly, D., Duarte, J.D., Kaiser, A., Schwabe, P., Varner, K., Westerbaan, B.: X-wing: The hybrid KEM you've been looking for. Cryptology ePrint Archive, Report 2024/039 (2024), https://eprint.iacr.org/2024/039
6. Barker, E., Chen, L., Regenscheid, A., Moody, D., Newhouse, W., Kent, K., Barker, W., Cooper, D., Souppaya, M., Housley, R., Turner, S.: Considerations for Achieving Crypto Agility: Strategies and Practices (2025). https://doi.org/10.6028/nist.cswp.39, http://dx.doi.org/10.6028/NIST.CSWP.39
7. Bitcoin Community: Bip-360: Pay to tapscript hash (p2tsh). Bitcoin Improvement Proposal (Draft), https://bip360.org/bip360.html
8. Buchmann, J., Dahmen, E., Ereth, S., Hülsing, A., Rückert, M.: On the security of the winternitz one-time signature scheme. In: Africacrypt 2011. LNCS, vol. 6737, pp. 363–378. Springer, Heidelberg (2011), https://eprint.iacr.org/2011/191.pdf
9. Drake, J., et al.: Hash-based multi-signatures for post-quantum ethereum. Cryptology ePrint Archive, Paper 2025/055 (2025), https://eprint.iacr.org/2025/055
10. Ethereum Community: Eip-4337: Account abstraction using alt mempool. Ethereum Improvement Proposal (2023), https://eips.ethereum.org/EIPS/eip-4337
11. Ethereum Foundation Privacy & Scaling Explorations: Post-quantum cryptography and ethereum (2025), https://pse.dev/projects/post-quantum-cryptography
12. Ethereum Research: Poqeth: Efficient post-quantum signature verification on ethereum (2024), https://ethresear.ch/t/poqeth-efficient-post-quantum-signature-verification-on-ethereum/21554
13. Ethereum Research: Tasklist for post-quantum eth (2024), https://ethresear.ch/t/tasklist-for-post-quantum-eth/21296
14. Fukuda, K., Matsuo, S.: Post-quantum blockchain: Transition landscape amidst evolving complexity. Cryptology ePrint Archive, Paper 2025/1626 (2025), https://eprint.iacr.org/2025/1626
15. Gentry, C., Peikert, C., Vaikuntanathan, V.: Trapdoors for hard lattices and new cryptographic constructions. In: STOC 2008. pp. 197–206. ACM (2008), https://eprint.iacr.org/2007/432
16. Giechaskiel, I., Cremers, C.J.F., Rasmussen, K.B.: On bitcoin security in the presence of broken cryptographic primitives. pp. 201–222 (2016). https://doi.org/10.1007/978-3-319-45741-3_11

17. Lopp, J., Papathanasiou, C.: Post quantum migration and legacy signature sunset. Bitcoin Improvement Proposal (Draft) (2025), https://qbip.org/
18. Meng, L., Chen, L.: A blockchain-based long-term time-stamping scheme. pp. 3–24 (2022). https://doi.org/10.1007/978-3-031-17140-6_1
19. Sato, M., Matsuo, S.: Long-term public blockchain: Resilience against compromise of underlying cryptography. In: 2017 26th International Conference on Computer Communication and Networks (ICCCN). p. 1–8. IEEE (Jul 2017). https://doi.org/10.1109/icccn.2017.8038516, http://dx.doi.org/10.1109/ICCCN.2017.8038516
20. The Quantum Insider: Solana takes a step toward pqc era with quantum-resistant vault (2025), https://thequantuminsider.com/2025/01/04/solana-takes-a-step-toward-pqc-era-with-quantum-resistant-vault/